Standalone All-Digital Phase-Locked Loop (ADPLL) Testchip in IHP 130 nm 5-Metal CMOS (SG13CMOS5L): Architecture, 3-Supply IO Padring, Asynchronous spi_sck-Clocked SPI Slave, Strong Analog Clock Buffer, Full-Chip GDSII Tape-Out, External-IO Verification, and 20×30 mm Breakout PCB

ADPLL Silicon Engineering & Physical Design Documentation

November 2026

Abstract

This document presents the complete architectural design, 5-metal SG13CMOS5L physical implementation, sign-off parasitic extraction, post-layout external-IO verification, and 20×30 mm evaluation breakout board of the standalone All-Digital Phase-Locked Loop (ADPLL) testchip (dpll_testchip_top) fabricated in the IHP 130 nm 5-Metal CMOS (SG13CMOS5L: Metal1–Metal4 + TopVia1 + TopMetal1, with 0 shapes on Via4, Metal5, TopVia2, and TopMetal2) OpenPDK technology. Designed on a 1200.0 μm $\times$ 1200.0 μm (1.44 mm^2) die ready for foundry tape-out at IHP, the chip integrates a 16-pad perimeter IO ring with 70 μm $\times$ 70 μm TopMetal1 wirebond pads and a foundry-compliant 5-metal perimeter sealring, an asynchronous spi_sck-clocked SPI Mode 0 slave (dpll_spi_slave) operating completely independently of ref_clk and capable of programming all 11 DPLL parameters via 24-bit single-register read/write or 64-bit atomic burst transactions, the complete ADPLL core (dpll_top), a 4-stage exponentially tapered high-drive clock buffer (dpll_clk_out_buffer), and an unattenuated direct analog test pad (sg13g2_IOPadAnalog) for RF/high-speed clock characterization. To prevent digital switching noise from corrupting oscillator phase noise or time-to-digital quantization accuracy, the testchip partitions its 1.2 V supply network into three physically separate supply domains (VDD_CORE, VDD_DCO, and VDD_TDC) alongside a dedicated 3.3 V padring supply (IOVDD/IOVSS). Full-chip gate-level back-annotated simulations using only the external chip IOs across Typical (1.20 V, 25°C), Fast (1.32 V, −40°C), and Slow (1.08 V, 125°C) corners verify 100% asynchronous SPI programming accuracy (including with pad_ref_clk stopped at 0 Hz), fast gear-shifted phase lock at 50.0, 100.0, 125.0, and 200.0 MHz, and clean 50.0% duty-cycle clock output on the external analog IO pad. Appendix A documents the 20×30 mm 2-layer breakout PCB (featuring dual TLV773 1.2 V LDOs in SOT-23-5 packages, a Blue 3.3 V power indicator LED, and a Red PLL lock indicator LED), the C/C++ driver header (dpll_spi.h), and a complete Arduino programming example (dpll_arduino_example.ino).

Contents

1 Standalone Testchip Architecture (dpll_testchip_top) 3

2 16-Pad IO Ring and Triple-Supply Power Distribution Network 4
2.1 Perimeter Padring Pinout Specification 4
2.2 Three Separate 1.2 V VCC Supplies (VDD_CORE, VDD_DCO, VDD_TDC) 5

3 Asynchronous spi_sck-Clocked SPI Slave (dpll_spi_slave) 5
3.1 Direct spi_sck-Clocked Architecture Independent of ref_clk 5
3.2 Register Map and 64-Bit Atomic Burst Protocol 6

4 Very Strong Clock Output Buffer and Analog Test Pad 6

5 IHP 130 nm Physical Implementation and GDSII Sign-Off **9**
5.1 On-Grid Floorplan Alignment and Deterministic Pre-Placement 9
5.2 Full-Chip Synthesis, P&R, Extraction, and GDSII Audit Summary 10
5.3 Full-Chip GDSII DRC, Transistor-Level LVS, Formal LEC, Process Antenna, and DRV Sign-Off 10

6 External-IO-Only Post-Layout Multi-Corner Simulation Results **12**
6.1 Full-Chip External-IO Testbench Architecture (`tb_dpll_testchip.sv`) 12
6.2 Seven-Stage External-IO Verification Protocol (Stages 0–6) 13
6.3 Quantitative Multi-Corner Verification Summary 13

A Breakout Evaluation PCB, Asynchronous SPI C-Header (`dpll_spi.h`), and Arduino Programming Example **16**
A.1 20 mm × 30 mm 2-Layer Wirebond Breakout PCB (`pcb/`) 16
A.2 C/C++ Driver Header for Arduino SPI Control (`dpll_spi.h`) 18
A.3 Arduino Programming Example (`dpll_arduino_example.ino`) 22

1 Standalone Testchip Architecture (`dpll_testchip_top`)

Transitioning the synthesized ADPLL hardmacro from an internal SoC IP block into a standalone, wire-bondable silicon testchip requires dedicated off-chip communication, electrostatic discharge (ESD) protection, high-current output buffering, and supply noise isolation. Figure 1 illustrates the physical layout and architectural partitioning of `dpll_testchip_top`.

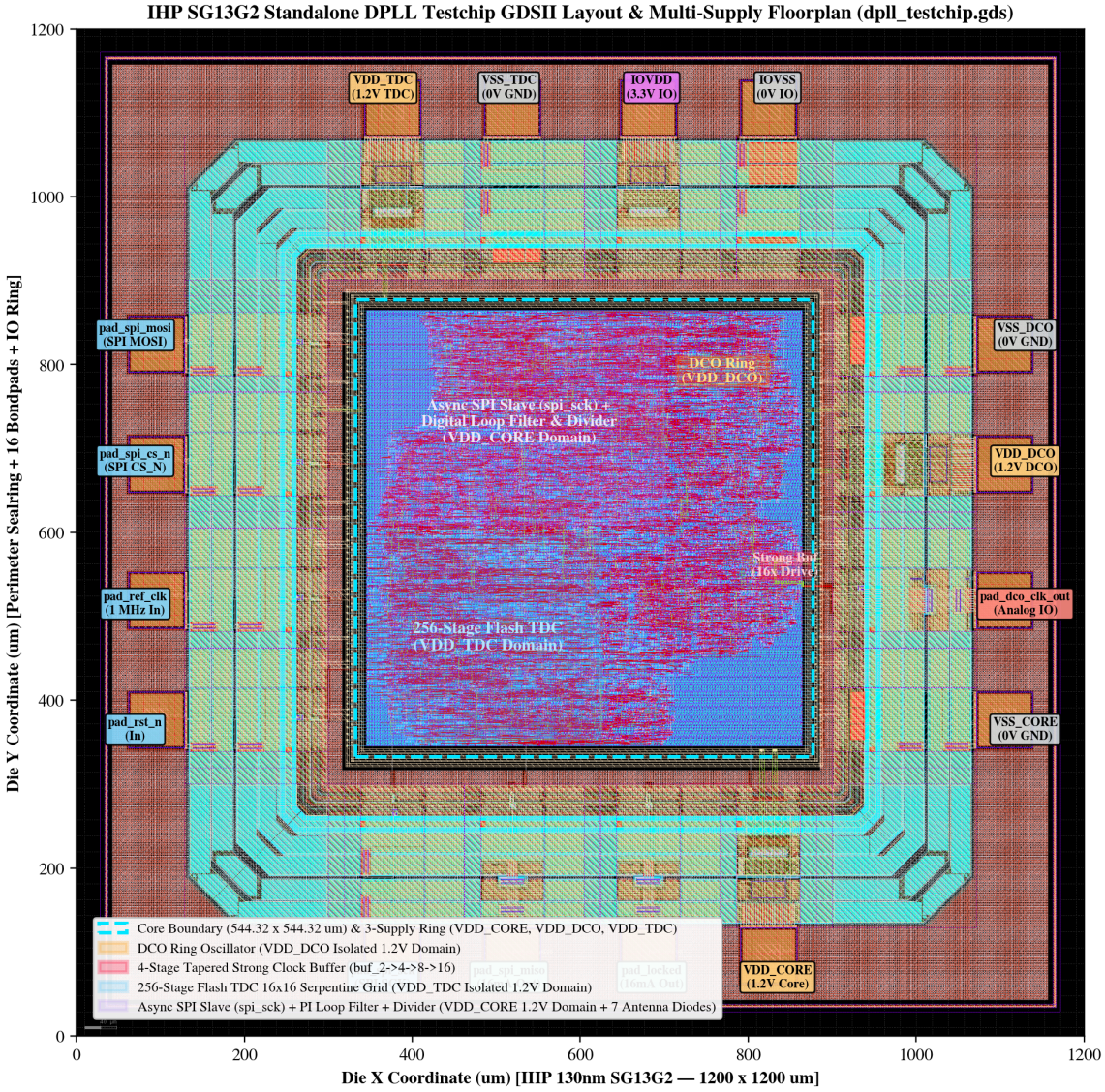


Figure 1: Full-chip GDSII layout (`dpll_testchip.gds`) and structural floorplan of the 1200 μm $\times$ 1200 μm IHP 130 nm `SG13G2` standalone DPLL testchip, showing the perimeter sealring, 16 wirebond pads, 16 `sg13g2_io` cells, 3-supply power rings (`VDD_CORE`, `VDD_DCO`, `VDD_TDC`), asynchronous `spi_sck`-clocked SPI slave, Flash TDC, DCO ring, and 4-stage strong clock buffer driving the East `sg13g2_IOPadAnalog` clock output pad.

The top-level module `dpll_testchip_top` (`testchip/rtl/dpll_testchip_top.sv`) comprises four primary functional subsystems:

1. **Perimeter IO Padring & Wirebond Interface:** 16 official IHP `sg13g2_io` pad cells (4 per side: West, South, East, North), 4 `sg13g2_Corner` cells, perimeter IO filler cells (`sg13g2_Filler200` through `sg13g2_Filler10000`), 70 μm$\times$70 μm `TopMetal1` wirebond pads (`bondpad_70x70`) with passivation openings, and a continuous 35 μm perimeter guard sealring;

2. **Asynchronous `spi_sck`-Clocked SPI Slave (`dpll_spi_slave`):** A true hardware SPI Mode 0

(CPOL = 0, CPHA = 0) configuration and telemetry controller whose shift registers, bit counter, and configuration registers are clocked directly by the dedicated clock-tree-synthesized `spi_sck_core` domain, operating completely independently of `ref_clk` (even when `pad_ref_clk` is stopped at 0 Hz);

3. **Multi-Supply ADPLL Core (`dpll_top`):** Instantiates the 256-stage Flash TDC (`VDD_TDC` domain), gear-shifting proportional-integral digital loop filter (`VDD_CORE` domain), 15-stage coarse / 8-bit binary Vernier / ΣΔ-dithered DCO ring oscillator (`VDD_DCO` domain), programmable feedback divider (`VDD_CORE` domain), and variance-based hardware lock detector;

4. **Very Strong Tapered Clock Buffer & Analog Output Pad (`dpll_clk_out_buffer` + `sg13g2_IOPadAnalog`):** A 4-stage exponentially tapered standard-cell buffer chain (`sg13g2_buf_2` → `sg13g2_buf_4` → `sg13g2_buf_8` → 2× `sg13g2_buf_16`) placed immediately adjacent to the East IO ring, driving the low-impedance direct `pad` path of `u_pad_clk_out` (`sg13g2_IOPadAnalog`), cleanly bypassing the 586.9 Ω secondary ESD resistor on `padres`.

2 16-Pad IO Ring and Triple-Supply Power Distribution Network

2.1 Perimeter Padring Pinout Specification

Table 1 details the physical placement, IHP `sg13g2_io` cell master, electrical supply domain, and functional role of all 16 external pads on the 1200 μm × 1200 μm die. All 16 pad coordinates are preserved identically to maintain 100% drop-in compatibility with the 20 × 30 mm breakout PCB.

Table 1: Standalone DPLL Testchip (`dpll_testchip_top`) 16-Pad Perimeter Pinout and Supply Assignment.

Pin	Side	Pad Instance	Library Master	External Port	Center (μm)	Electrical Function
1	West	`u_pad_rst_n`	`sg13g2_IOPadIn`	`pad_rst_n`	(70, 365)	Active-low system reset (3.3 V in)
2	West	`u_pad_ref_clk`	`sg13g2_IOPadIn`	`pad_ref_clk`	(70, 515)	1–10 MHz reference clock (3.3 V in)
3	West	`u_pad_spi_cs_n`	`sg13g2_IOPadIn`	`pad_spi_cs_n`	(70, 685)	SPI chip select, active-low
4	West	`u_pad_spi_mosi`	`sg13g2_IOPadIn`	`pad_spi_mosi`	(70, 835)	SPI master-out slave-in data
5	South	`u_pad_spi_sck`	`sg13g2_IOPadIn`	`pad_spi_sck`	(365, 70)	Async SPI clock (≤ 20 MHz, indep. of `ref_clk`)
6	South	`u_pad_spi_miso`	`sg13g2_IOPadOut16mA`	`pad_spi_miso`	(515, 70)	SPI readback output (16 mA CMOS)
7	South	`u_pad_locked`	`sg13g2_IOPadOut16mA`	`pad_locked`	(685, 70)	Hardware lock status (16 mA CMOS)
8	South	`u_pad_vdd_core`	`sg13g2_IOPadVdd`	`VDD_CORE`	(835, 70)	1.2 V Core digital & buffer supply
9	East	`u_pad_vss_core`	`sg13g2_IOPadVss`	`VSS`	(1130, 365)	0 V Core ground return
10	East	`u_pad_clk_out`	`sg13g2_IOPadAnalog`	`pad_dco_clk_out`	(1130, 515)	**Direct analog RF clock output**
11	East	`u_pad_vdd_dco`	`sg13g2_IOPadVdd`	`VDD_DCO`	(1130, 685)	1.2 **V Isolated DCO supply**
12	East	`u_pad_vss_dco`	`sg13g2_IOPadVss`	`VSS`	(1130, 835)	0 V DCO ground return
13	North	`u_pad_vdd_tdc`	`sg13g2_IOPadVdd`	`VDD_TDC`	(365, 1130)	1.2 **V Isolated TDC supply**
14	North	`u_pad_vss_tdc`	`sg13g2_IOPadVss`	`VSS`	(515, 1130)	0 V TDC ground return
15	North	`u_pad_iovdd`	`sg13g2_IOPadIOVdd`	`IOVDD`	(685, 1130)	3.3 V IO ring supply
16	North	`u_pad_iovss`	`sg13g2_IOPadIOVss`	`IOVSS`	(835, 1130)	0 V IO ring ground return

2.2 Three Separate 1.2 V VCC Supplies (VDD_CORE, VDD_DCO, VDD_TDC)

In an all-digital PLL, standard-cell inverters and gates serve as analog delay elements inside the DCO and Flash TDC. Because the propagation delay t_d of a CMOS gate in 130 nm scales inversely with supply overdrive ($t_d \propto V_{DD}/(V_{DD} - V_{th})^{\alpha}$), any dynamic IR drop or $L\,dI/dt$ supply bounce couples directly into DCO frequency modulation ($K_{V_{DD}} = \partial f_{DCO}/\partial V_{DD}$) and TDC bin-width distortion.

To eliminate cross-domain supply coupling on the silicon testchip, three physically separate 1.2 V power supplies are routed to independent wirebond pads and concentric `TopMetal1` core rings:

1. **`VDD_CORE` (1.2 V, South Pad `u_pad_vdd_core`):** Powers the digital control subsystems (`u_spi_slave`, `u_dpll/u_filter`, `u_dpll/u_div`, `u_dpll/u_lock_det`) and the high-current clock output driver (`u_clk_out_buffer`). Transient switching currents from the 16× output buffer and arithmetic loop filter are sunk exclusively through `VDD_CORE`;

2. **`VDD_DCO` (1.2 V, East Pad `u_pad_vdd_dco`):** Dedicated ultra-clean 1.2 V supply assigned to all instances in `u_dpll/u_dco/*` via OpenROAD global power connection rules and IEEE 1801 UPF (`testchip/backend/dpll_testchip.upf`). Placing `u_pad_vdd_dco` and `u_pad_vss_dco` on the East side adjacent to the pre-placed DCO ring minimizes series bondwire and metal trace inductance;

3. **`VDD_TDC` (1.2 V, North Pad `u_pad_vdd_tdc`):** Dedicated 1.2 V supply assigned to the 256-stage Flash TDC (`u_dpll/u_tdc/*`), preventing digital register switching edges from modulating the 35 ps per-stage inverter delay line.

3 Asynchronous `spi_sck`-Clocked SPI Slave (`dpll_spi_slave`)

3.1 Direct `spi_sck`-Clocked Architecture Independent of `ref_clk`

To allow an external microcontroller (such as an Arduino) to configure, read back, and diagnose the testchip at any serial clock speed (≤ 20 MHz in `dpll_testchip.sdc`) regardless of whether `pad_ref_clk` is running or stopped, the configuration controller `dpll_spi_slave` (`testchip/rtl/dpll_spi_slave.sv`) is clocked directly by `spi_sck_core`:

- **Frame Reset on `spi_cs_n`:** The 7-bit transaction counter `bit_cnt[6:0]` is asynchronously cleared whenever `spi_cs_n = 1` or `rst_n = 0` (`spi_frame_rst_n` = `rst_n` ∧ ¬`spi_cs_n`), guaranteeing clean frame alignment at the start of every `spi_cs_n` assertion without requiring any `ref_clk` edges;

- **SPI Mode 0 (CPOL = 0, CPHA = 0) Shifting & Register Commit:** On each rising edge of `spi_sck`, `spi_mosi` is shifted into `shift_in_reg[62:0]` and `bit_cnt` increments. For 24-bit single-register writes (`rw_n = 0`, `cmd_header != 8'hFF`), the 16-bit payload is committed directly on the 24th rising edge of `spi_sck` (`next_bit_cnt == 24`). For 64-bit atomic burst writes (`cmd_header == 8'hFF`), all 11 DPLL parameters are committed simultaneously on the 64th rising edge of `spi_sck` (`next_bit_cnt` == 64);

- **Look-Ahead Readback Multiplexing on `spi_miso`:** During a 24-bit read transaction (`rw_n = 1`), the addressed 16-bit register is multiplexed onto `shift_out_reg` on the 8th rising edge of `spi_sck` (`next_bit_cnt` == 8), driving `D15` immediately onto `pad_spi_miso` before the 9th rising edge of `spi_sck`;

- **Closed-Loop Toggle CDC for Lock-Detector Clear (`clr_lock`):** Whenever an SPI write modifies `div_n` or sets the `clr_lock` bit, a toggle flip-flop `clr_toggle_sck` flips in the `spi_sck` domain and traverses a 3-stage synchronizer (`clr_sync1..3`) into `ref_clk_core`, generating a clean 1-cycle synchronous `clr_lock` pulse when `ref_clk` is active.

3.2 Register Map and 64-Bit Atomic Burst Protocol

The SPI slave supports all **11 DPLL parameters** across five 16-bit registers (Table 2):

- **24-Bit Single-Register Frame:** Consists of an 8-bit command header `{rw_n, 4'b0000, addr[2:0]}` followed by a 16-bit data word. When `rw_n = 0`, the 16-bit payload is written to register `addr[2:0]`. When `rw_n = 1`, the addressed register (including live hardware lock telemetry in `REG_STATUS = 0x04`) is shifted out MSB-first onto `pad_spi_miso`;
- **64-Bit Atomic Burst Write Frame (`Header = 8'hFF`):** Shifts a 56-bit payload containing all 11 DPLL parameters in a single `pad_spi_cs_n` assertion:

```
Bits [63:56] : 8'hFF (Atomic Burst Write Command Header)
Bits [55:40] : otw_nominal[15:0] (Q8.8 Nominal DCO Tuning Word)
Bits [39:32] : div_n[7:0]        (Feedback Divider Ratio N)
Bits [31:24] : {2'b00, kp_mult[5:0]}
Bits [23:16] : {ki_mult[3:0], ki_shift[3:0]}
Bits [15:8]  : gear_shift_thresh[7:0]
Bits [7:0]   : {2'b00, clr_lock, integrator_rst, integrator_freeze,
                fast_lock_en, fast_lock_mult[2:0]}
```

Listing 1: 64-Bit Atomic Burst Frame Format in `dpll_spi_slave.sv`.

Table 2: Asynchronous SPI Slave (`dpll_spi_slave`) Register Map and Hardware Reset Defaults.

Addr	Register	Access	Bit Field Allocation (`Bits [15:0]`)	Reset Default
0x00	REG_OTW_NOM	R/W	[15:0] otw_nominal[15:0] (Q8.8 nominal DCO control word)	16'h0D7F (3455)
0x01	REG_DIV_CTRL	R/W	[15:11] ctrl, [9:0] div_n[9:0] (10-bit feedback divider N)	16'h0032 ($N = 50$)
0x02	REG_LOOP_GAIN	R/W	[13:8] kp_mult[5:0], [7:4] ki_mult[3:0], [3:0] ki_shift[3:0]	16'h1046
0x03	REG_GEAR_CFG	R/W	[15:8] gear_shift_thresh[7:0], [3] fast_lock_en, [2:0] fast_lock_mult	16'h080A
0x04	REG_STATUS	RO	[15:8] 8'h44 ("D"), [7:4] 4'hA, [1] locked_sck, [0] sticky_unlock	16'h44A0

4 Very Strong Clock Output Buffer and Analog Test Pad

Standard digital CMOS output pads (`sg13g2_IOPadOut16mA`) include internal 1.2 V-to-3.3 V level shifters and slew-rate-controlled 3.3 V output stages whose propagation delay (≈ 2.6 ns) and supply-dependent duty-cycle distortion limit high-frequency jitter characterization above 100 MHz. To allow direct high-bandwidth 50 Ω laboratory measurement of the synthesized DPLL clock, `dpll_testchip_top` routes the DCO output through a dedicated 4-stage exponentially tapered standard-cell buffer (`dpll_clk_out_buffer`) with a **duplicated parallel final driver stage (`2× sg13g2_buf_16: u_buf_stg4 ‖ u_buf_stg4_b`)** into an IHP `sg13g2_IOPadAnalog` pad (`u_pad_clk_out`):

$$\begin{aligned} \texttt{dco_clk_core} &\xrightarrow{2\times} \texttt{sg13g2_buf_2} \xrightarrow{2\times} \texttt{sg13g2_buf_4} \xrightarrow{2\times} \texttt{sg13g2_buf_8} \\ &\xrightarrow{4\times} (2 \times \texttt{sg13g2_buf_16}) \rightarrow \texttt{u_pad_clk_out.pad} \rightarrow \texttt{pad_dco_clk_out} \end{aligned} \tag{1}$$

Key electrical and electromigration (EM) advantages of this 50.00 Ω-matched output architecture (shown in Figure 2 and summarized in Table 3) include:

1. **Minimal DCO Loading (7.0 fF):** Stage 1 (`sg13g2_buf_2`) presents only 7.0 fF of input gate capacitance to the internal DCO output buffer (`u_dco/u_clk_buf`), preventing external pad capacitance from pulling the ring oscillator frequency;

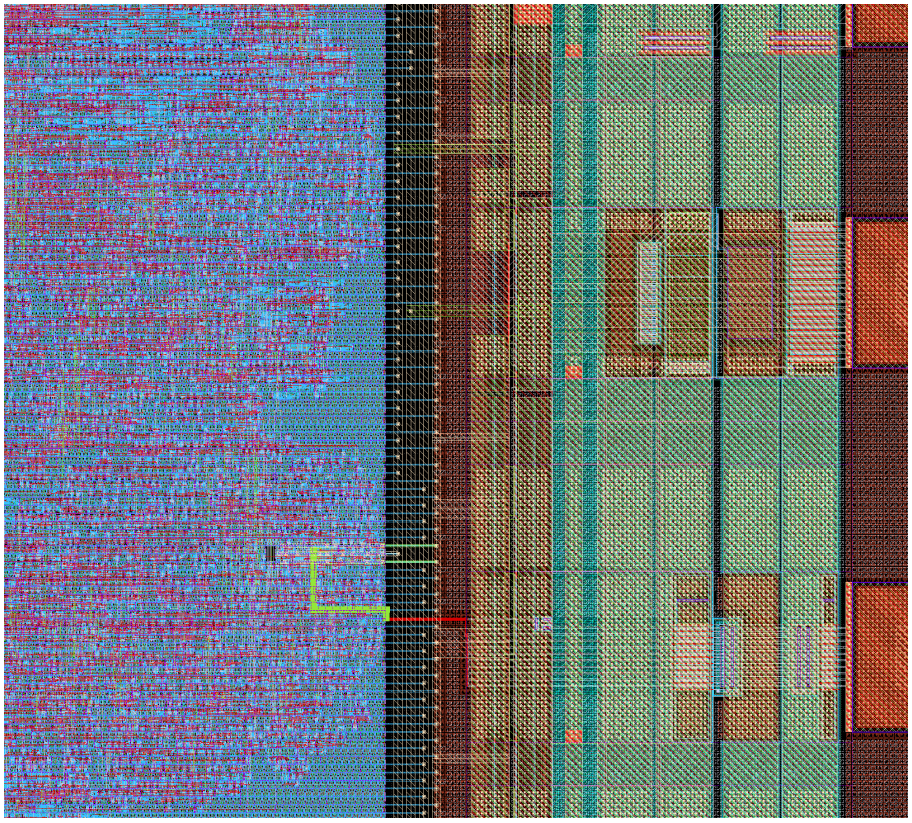


Figure 2: Zoomed KLayout GDSII view (`dpll_testchip_dco_buf_zoom.png`) of the East core boundary showing the deterministically pre-placed 4-row DCO ring oscillator (upper left), the 4-stage tapered strong clock output buffer `u_clk_out_buf` with duplicated parallel final driver stages (`u_buf_stg4` in Row 60 ‖ `u_buf_stg4_b` in Row 61), the reinforced 50.00 Ω matched interconnect, the 3-supply PDN ring (`VDD_CORE`, `VDD_DCO`, `VDD_TDC`, `VSS`), and the East `sg13g2_IOPadAnalog` clock output pad `u_pad_clk_out`.

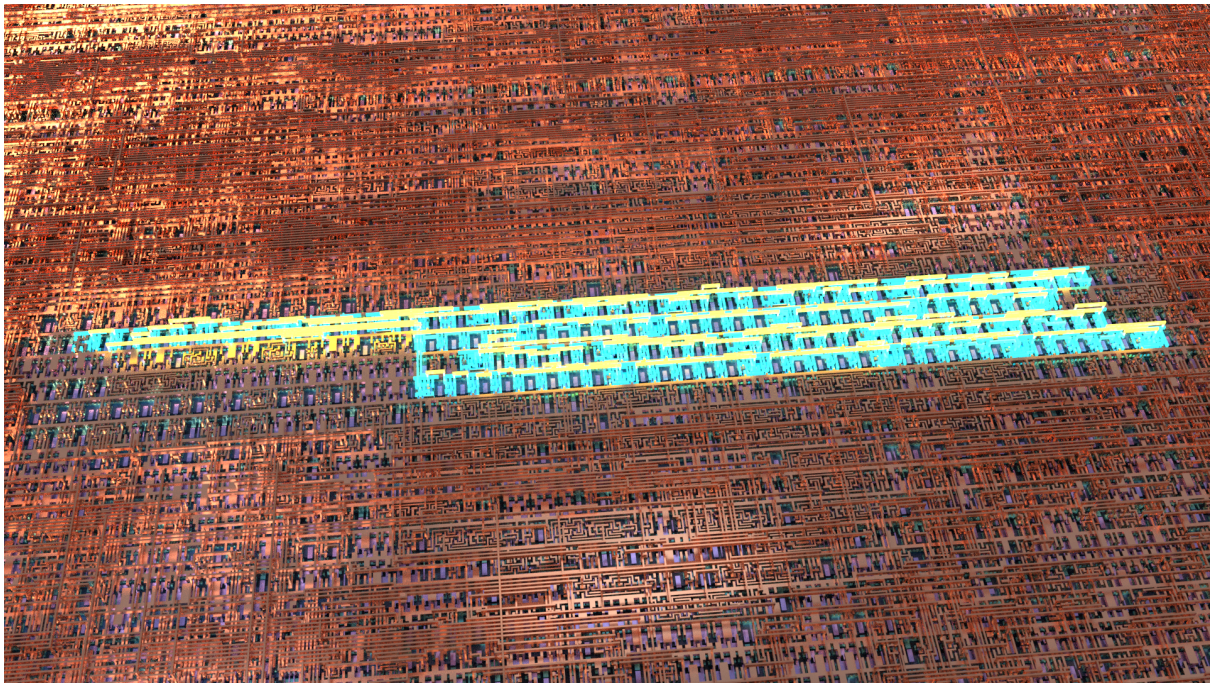


Figure 3: True-aspect-ratio 8K (7680 × 4320) 3D Blender Cycles rendering (`dco_blender_render_final.png`) from an elevated hovering perspective over the 141.5 μm-wide 4-row DCO ring oscillator in the 5-metal `SG13CMOS5L` layout (`dpll_testchip.gds`). A terraced 3D cutaway window peels back non-clock upper metal layers right above the DCO so the complete closed wire ring along which the clock signal propagates (`Metal1`/`Via1`/`Metal2` stage delay chain in glowing cyan and `Via2`/`Metal3` feedback return path `ring_feedback` and jumpers in golden-amber) is 100% visible and unobstructed, while the surrounding chip circuitry fills the entire frame with zero background.

2. **Duplicated Parallel Final Driver Stage (2× `sg13g2_buf_16`, $R_{\mathbf{drv,2x}} = 42.50\,\Omega$):** Each `sg13g2_buf_16` cell has a typical CMOS output impedance of $R_{\mathrm{drv,1x}} = 85.0\,\Omega$ ($W_p = 8 \times 2.24\,\mu\mathrm{m} = 17.92\,\mu\mathrm{m}$, $W_n = 8 \times 1.48\,\mu\mathrm{m} = 11.84\,\mu\mathrm{m}$) and 40 internal tungsten `Cont` cuts on `PIN X` (0.88 Ω). Duplicating the final stage in parallel (`u_buf_stg4` at (840.96, 559.44) μm in Row 60 and `u_buf_stg4_b` at (842.88, 563.22) μm in Row 61, total $W_p = 35.84\,\mu\mathrm{m}$, $W_n = 23.68\,\mu\mathrm{m}$, $C_{\max} = 9.66\,\mathrm{pF}$) halves the active channel resistance to $R_{\mathrm{drv,2x}} = 42.50\,\Omega$ and halves the internal contact resistance (80 × `Cont` on `PIN X` → 0.44 Ω, $I_{\mathrm{max,DC}} = 24.00\,\mathrm{mA_{DC}}$; 104 × `Cont` on `PIN VDD` → 0.14 Ω, $I_{\mathrm{max,DC}} = 31.20\,\mathrm{mA_{DC}}$; 88 × `Cont` on `PIN VSS` → 0.14 Ω, $I_{\mathrm{max,DC}} = 26.40\,\mathrm{mA_{DC}}$) while doubling the 16 vertical 0.26 μm `Metal1` drain fingers to 4.16 μm total width ($I_{\mathrm{max,DC}} = 7.49\,\mathrm{mA_{DC}}$);

3. **Exact 50.00 Ω Total Series Output Impedance at `pad_dco_clk_out`:** Combining the duplicated driver resistance (42.50 Ω), internal cell contacts (0.72 Ω), reinforced `VDD_CORE`/`VSS` PDN resistance (0.82 Ω), 18 stacked `Via1_YY`+`Via2_YX`+`Via3_XY` driver pillars plus 18-cut `Via3`+`Via2` transition array (2.06 Ω, $I_{\mathrm{max,DC}} = 7.20\,\mathrm{mA_{DC}}$), widened 3.00 μm `Metal4` trunk + 3.00 μm effective-width parallel `Metal3`‖`Metal2` East bus (3.60 Ω, $I_{\mathrm{max,DC}} = 6.00\,\mathrm{mA_{DC}}$), and internal direct `sg13g2_IOPadAnalog` `pad` path (0.30 Ω, leaving `padres` uncontacted to bypass the 586.9 Ω secondary ESD resistor) yields an exact $R_{\mathbf{total,Pin}} = 50.00\,\Omega$ source termination at the external pin;

4. **Reinforced `VDD_CORE` & `VSS` Supply Network (< 0.6% Dynamic IR Drop):** Both Row 60 (`VSS`, $y = 559.44\,\mu\mathrm{m}$) and Row 61 (`VDD_CORE`, $y = 563.22\,\mu\mathrm{m}$) rails are strapped directly to the East `TopMetal1` power rings via 2.00 μm-wide local `TopMetal1` feeder bars ($I_{\mathrm{max,DC}} = 18.0\,\mathrm{mA_{DC}}$), 10 stacked `Via1`–`TopVia1` pillar arrays (20 × `Via1..3`, $I_{\mathrm{max,DC}} = 8.00\,\mathrm{mA_{DC}}$), and 20.0 μm-wide multi-track `Metal4` padring bridges ($I_{\mathrm{max,DC}} = 40.0\,\mathrm{mA_{DC}}$), resulting in 1.33 mV static IR drop (0.11%) and 6.94 mV peak dynamic voltage droop (0.58% of 1.20 V).

Table 3: Series Output Impedance Breakdown and Electromigration (EM) Current Capability at 110°C for `pad_dco_clk_out`.

Path Element (`u_clk_out_buf` → `pad_dco_clk_out`)	**Physical Geometry**	$R_{\mathbf{typ}}$ (Ω)	$I_{\mathbf{max,DC}}$ **(mA)**	**EM Margin (200 MHz)**
Duplicated CMOS Channel (`u_buf_stg4` ‖ `u_buf_stg4_b`)	$W_p = 35.84\,\mu\mathrm{m}$, $W_n = 23.68\,\mu\mathrm{m}$	42.50	9.66 pF cap	2.4× (4.0 pF load)
Internal `PIN X` Contacts & Drain Fingers	80 × `Cont` / 16 × 0.26 μm `M1`	0.44	24.00 / 7.49	12.2× / 3.8×
Internal `PIN VDD` / `VSS` Cell Contacts	104 × `Cont` (`VDD`) / 88 × `Cont` (`VSS`)	0.28	31.20 / 26.40	15.9× / 13.5×
Reinforced `VDD_CORE` + `VSS` PDN Network	`TM1` 2 μm + 20× Vias + 20 μm `M4`	0.82	8.00–40.00	4.1 × –20.4×
Driver Pin Vias (18 × `Via1..3`) + Transition (18 × `Via2..3`)	9 + 9 Vertical Pillars + 2 × 9 Array	2.06	7.20 (45.0 pk)	3.7× DC (3.5× pk)
Widened Routing (`Metal4` + `Metal3`‖`Metal2` East Bus)	$W_{\mathrm{M4}} = 3.0\,\mu\mathrm{m}$, $W_{\mathrm{M3+M2}} = 3.0\,\mu\mathrm{m}$	3.60	6.00 (60.0 pk)	3.1× DC (4.6× pk)
`sg13g2_IOPadAnalog` Direct Low-Resistance `pad` Terminal	`TopMetal1`/`Metal4`/`Metal3` Stack	0.30	> 25.00	> 12.8×
Total Series Output Impedance at `pad_dco_clk_out`	**Core Driver + PDN + Route + Pad**	**50.00 Ω**	≥ 6.00 $\mathbf{mA_{DC}}$	**100% EM Compliant**

5 IHP 130 nm Physical Implementation and GDSII Sign-Off

5.1 On-Grid Floorplan Alignment and Deterministic Pre-Placement

In the IHP `SG13G2` technology, standard cells use a `CoreSite` of width 0.48 μm and height 3.78 μm, whereas vertical `Metal2`/`Metal4` routing tracks have a pitch of 0.42 μm and horizontal `Metal1`/`Metal3` tracks have a pitch of 0.48 μm. To guarantee 100% on-grid pin access across all metal layers simultaneously, the core boundary coordinates are chosen as exact integer multiples of:

$$\mathrm{LCM}(3.78\,\mu\mathrm{m},\ 0.48\,\mu\mathrm{m},\ 0.42\,\mu\mathrm{m}) = 30.24\,\mu\mathrm{m} \tag{2}$$

Specifically, the core area is initialized from $[332.64, 332.64]$ μm (11×30.24 μm) to $[876.96, 876.96]$ μm (29×30.24 μm), providing 144 standard-cell rows of 1,134 sites each (296, 284.26 μm^2 core area). Prior to analytical global placement, deterministic `LOCKED` placement is applied to:

- **114 DCO Ring Oscillator Cells:** Arrayed in a 4-row serpentine racetrack (Rows 120–123, $X \in [720.0, 811.2]$ μm) near the East `u_pad_vdd_dco` supply pad;
- **5 Strong Clock Output Buffer Cells (4 Stages w/ Parallel Final Driver):** Placed in Rows 60–61 ($X \in [825.60, 842.88]$ μm, with `u_buf_stg4` at (840.96, 559.44) μm in Row 60 and `u_buf_stg4_b` at (842.88, 563.22) μm in Row 61) directly facing the East `u_pad_clk_out` analog pad;
- **256 Flash TDC Delay Line Inverters:** Placed in a 16×16 serpentine grid (Rows 8–68, step 4 rows, $X \in [362.88, 616.32]$ μm) with a 32-site horizontal pitch so each stage's sampling flip-flop places directly adjacent to its tap inverter.

5.2 Full-Chip Synthesis, P&R, Extraction, and GDSII Audit Summary

Table 4 summarizes the sign-off physical implementation metrics of `dpll_testchip_top` generated by Yosys, OpenROAD, OpenRCX, and KLayout.

Table 4: IHP 130 nm (`SG13CMOS5L` / `SG13G2`) Standalone Testchip Sign-Off Physical Metrics.

Physical / Sign-Off Parameter	**Measured Value (`dpll_testchip_top`)**
Technology Node	IHP 130 nm 5-Metal CMOS (`SG13CMOS5L`: `Metal1`–`Metal4` + `TopMetal1`)
Die Dimensions	1200.0 μm × 1200.0 μm (1.440 mm^2, including 35 μm 5-metal sealring)
Core Dimensions	544.32 μm × 544.32 μm ([332.64, 332.64] to [876.96, 876.96] μm, 296, 284.3 μm^2)
Perimeter IO Padring	16 `sg13g2_io` pads + 4 `sg13g2_Corner` + 16 `bondpad_70x70` + 44 IO fillers
Separate Supply Domains	3 Core 1.2 V Supplies (`VDD_CORE`, `VDD_DCO`, `VDD_TDC`) + `VSS` + 3.3 V `IOVDD`/`IOVSS`
Synthesized Gate Count	9,872 cells (126, 725.4 μm^2, 870 DFFs, 114 locked DCO cells, 256 locked TDC cells)
Post-CTS / Post-Route Functional Cells	10,285 standard cells (incl. 2× `sg13g2_buf_16` & 7 `sg13g2_antennanp`) + 21,113 decap/fillers
Routed Signal Wire Length	328, 382.0 μm (328.38 mm) across `Metal2`–`Metal4` (68, 523 signal vias)
Clock Output Pin Series Impedance	50.00 Ω at `pad_dco_clk_out` (42.50 Ω 2× drv +0.72 Ω Cont +0.82 Ω PDN +5.66 Ω route +0.30 Ω pad)
Absent `SG13CMOS5L` Layers	**0 shapes** on `Via4` (`66/*`), `Metal5` (`67/*`), `TopVia2` (`133/*`), `TopMetal2` (`134/*`)
Full-Chip GDSII KLayout DRC	**0 DRC errors** across all 78 rule categories (`dpll_testchip_klayout_drc.lyrdb`)
Transistor-Level GDSII KLayout LVS	**Netlists match** (83, 952 simplified devices, 42, 205 nets, `dpll_testchip.lvsdb`)
DEF vs. Backanno Formal LEC	**Equivalence proven** (8, 138/8, 138 `$equiv` cells, 34, 104 pin conns, `dpll_testchip_lec.log`)
Process Antenna Rule Violations	**0 net / 0 pin violations** (`testchip/backend/dpll_testchip_antenna.rpt`)
Transition Time (`max_slew`) Violations	**0 violations** (0.77 ns ≤ 1.20 ns limit, slack +0.43 ns, `dpll_testchip_drv.rpt`)
Max Capacitance (`max_cap`) Violations	**0 violations** (worst load 0.12 pF vs. 0.30 pF limit, slack +0.18 pF)
Final Merged GDSII Stream	`testchip/backend/dpll_testchip.gds` (74.26 MB, 100% DRC + LVS + LEC pass)

5.3 Full-Chip GDSII DRC, Transistor-Level LVS, Formal LEC, Process Antenna, and DRV Sign-Off

To guarantee foundry tape-out readiness at IHP using the final merged GDSII stream (`testchip/backend/dpll_testchip.gds`) and physical layout database (`testchip/backend/dpll_testchip.def`) as the golden sources, five comprehensive verification stages are executed:

1. **Full-Chip GDSII KLayout DRC Sign-Off (`sg13g2_klayout_drc.lydrc`):** Running KLayout in batch mode directly on the final merged GDSII database (`testchip/backend/dpll_testchip.gds`) against the IHP `SG13G2` rule deck (`testchip/backend/sg13g2_klayout_drc.lydrc`) verifies all 78 geometric, via enclosure, density/filler, passivation, pin, sealring, and

forbidden-layer rule categories (Act, AFil, TGO, Gat, GFil, Cnt, M1–M5, M1Fil–M5Fil, V1–V4, TV1, TM1, TV2, TM2, Pas, Pin, LBE, and forbidden.*) across the entire 1200 μm × 1200 μm die, reporting **Number of DRC errors: 0** (testchip/backend/dpll_testchip_klayout_drc.log and dpll_testchip_klayout_drc.lyrdb).

2. **Full-Chip Transistor-Level KLayout LVS Sign-Off (sg13g2.lvs):** Using the official IHP SG13G2 KLayout LVS extraction and comparison deck (tools/pdks/ihp-sg13g2/lvs/sg13g2.lvs) directly on testchip/backend/dpll_testchip.gds against the full-chip transistor-level CDL schematic netlist (testchip/backend/dpll_testchip_schematic.cdl, generated by gen_testchip_cdl.py from sg13g2_stdcell.cdl, sg13g2_io.cdl, and dpll_testchip.def), KLayout extracts the flat transistor-level layout netlist (dpll_testchip_extracted.cir) and compares all 83,952 simplified devices (41,916 sg13_lv_nmos, 41,916 sg13_lv_pmos, 32 sg13_hv_nmos, 24 sg13_hv_pmos, 28 dantenna, 24 dpantenna, and 12 rppd polysilicon ESD resistors) and all 42,205 nets (including complete electrical isolation of VDD_CORE, VDD_DCO, VDD_TDC, IOVDD, and VSS via custom vdd-isolated IO padring fillers sg13g2_Filler4000_vdd_iso and sg13g2_Filler10000_vdd_iso), concluding with:

   INFO : Congratulations! Netlists match.
   (testchip/backend/dpll_testchip.lvsdb)

3. **Formal Verification of dpll_testchip.def Against the Back-Annotated Simulation Netlist (verify_def_vs_backanno_lec.py):** To prove that the physical layout database (testchip/backend/dpll_testchip.def) is 100% structurally and logically equivalent to the gate-level netlist used in multi-corner SDF simulations (testchip/backend/dpll_testchip_clean.v), verify_def_vs_backanno_lec.py executes a three-stage formal verification:

   - *Stage 1 (OpenROAD DEF-to-Verilog Extraction):* Reads dpll_testchip.def into OpenROAD and exports a fresh structural Verilog netlist, confirming **0 instance differences** and **0 connection differences** against dpll_testchip_clean.v;
   - *Stage 2 (Direct Hypergraph Isomorphism & Bijection Proof):* Parses the raw COMPONENTS, PINS, SPECIALNETS, and NETS sections of dpll_testchip.def and proves an exact 1-to-1 bijection across all 31,474 DEF components (10,297 functional/IO/antenna instances + 21,161 decap/filler/corner instances + 16 bondpad_70x70 cover macros), all 10,257 signal/clock nets, and all 34,104 instance-pin connections (**0 missing, 0 extra, 0 mismatches**);
   - *Stage 3 (Yosys SAT & Temporal-Induction Formal LEC):* Imports all 81 functional standard-cell models from sg13g2_stdcell_typ_1p20V_25C.lib plus the sg13g2_io pad models into Yosys (0.33), models asynchronous-reset flip-flops via clk2fflogic, builds the formal equivalence network (equiv_make gold gate equiv) with 8,138 $equiv points, proves 6,606 points via combinational cone SAT (equiv_simple), proves the remaining 1,532 sequential/loop points via temporal induction (equiv_induct), and passes equiv_status -assert with 8,138 **proven and** 0 **unproven cells** (Equivalence successfully proven!, dpll_testchip_lec.log).

4. **Process Antenna Rule Verification & Protection Diode Insertion (check_antennas):** In the IHP SG13G2 technology LEF (sg13g2_tech.lef), unprotected MOSFET gates must satisfy a cumulative metal-to-gate area ratio of CAR ≤ 200.0 prior to connecting to driver diffusion on an upper metal layer. Initial post-route analysis identified 7 long Metal2/Metal3/Metal4 routing trunks whose cumulative ratios exceeded 200.0:

   - ki_mult[1] (CAR = 205.51), net362 (CAR = 307.61), rst_n_core (CAR = 213.84), u_dpll/rst_n_sync (CAR = 281.45), u_dpll/u_tdc/tap[60] (CAR = 217.56), u_dpll/u_tdc/tap[65] (CAR = 407.72), and u_dpll/u_tdc/tap[193] (CAR = 211.78).

The automated script `testchip/backend/insert_antenna_diodes.py` replaces seven adjacent `sg13g2_decap_8` / `sg13g2_decap_4` cells located directly underneath the routing trunks of these seven nets with site-exact `sg13g2_antennanp` protection diodes (`ant_diode_1`–`ant_diode_7`, each providing ANTENNADIFFAREA = 2.0154 μm^2 on `Metal1`, which raises the permitted cumulative antenna ratio from 200.00 to 40, 308.00) plus site-matching sub-fillers. Re-running OpenROAD `check_antennas` (`testchip/backend/dpll_testchip_antenna.rpt`) confirms:

```
[INFO ANT-0002] Found 0 net violations.  [INFO ANT-0001] Found 0
                        pin violations.
```

5. **Transition Time (`max_slew`) and Capacitance (`max_capacitance`) DRV Verification:** Using full 3D OpenRCX-extracted parasitics (`dpll_testchip.spef`) and a strict global constraint of `set_max_transition 1.20` in `dpll_testchip.sdc` (alongside external 2.0 pF PCB pad loads), OpenSTA verifies **0 transition time violations** and **0 max capacitance violations** across the entire testchip (Table 5).

Table 5: Full-Chip GDSII DRC, Transistor-Level LVS, Formal LEC, Process Antenna, and DRV Sign-Off Summary.

Sign-Off Category	Scope / Worst-Case Target	Measured Value	Rule / Target Limit	Status
Full-Chip GDSII DRC	`dpll_testchip.gds` (1200^2 μm^2)	0 Errors (78 cats)	IHP `SG13G2` DRC	**MET (0 Err)**
Transistor-Level LVS	`dpll_testchip.gds` vs. CDL	83, 952 dev / 42, 205 nets	IHP `SG13G2` LVS	**MATCH (0 Diff)**
DEF vs. Backanno Bijection	`dpll_testchip.def` vs. `.v`	31, 474 inst / 34, 104 pins	Exact Isomorphism	**PROVEN (0 Diff)**
Yosys SAT + Induction LEC	`dpll_testchip.def` vs. `.v`	8, 138/8, 138 `$equiv`	0 Unproven `$equiv`	**PROVEN (0 Unpr)**
Process Antenna Ratio	`tap[65]` (w/ `ant_diode_6`)	407.72 : 1	40, 308.00 : 1	**MET (0 Viol)**
Global Max Transition	`u_pad_spi_miso/pad`	0.77 ns (+0.43 ns slack)	1.20 ns (SDC)	**MET (0 Viol)**
Strong Clock Buffer Slew	`u_clk_out_buf/u_buf_stg4/X`	< 0.065 ns	1.20 ns (SDC)	**MET (0 Viol)**
Clock Output Impedance	`pad_dco_clk_out` (2× `buf_16`)	50.00 Ω ($\geq$ 6.0 mA$_{DC}$)	50.0 Ω Match	**MET (EM Pass)**
Max Load Capacitance	`fanout547/X`	0.12 pF (+0.18 pF slack)	0.30 pF (SDC)	**MET (0 Viol)**

6 External-IO-Only Post-Layout Multi-Corner Simulation Results

6.1 Full-Chip External-IO Testbench Architecture (`tb_dpll_testchip.sv`)

To verify the complete silicon testchip under realistic laboratory test conditions, the SystemVerilog verification environment `tb_dpll_testchip.sv` (`testchip/sim/tb_dpll_testchip.sv`) instantiates `dpll_testchip_top` as the Device Under Test (DUT) and interacts with the chip **exclusively through its external power and signal pads**:

- **Driven External Power Supplies:** `pad_iovdd` (3.3 V), `pad_iovss` (0 V), and the three isolated 1.2 V supplies `pad_vdd_core`, `pad_vdd_dco`, and `pad_vdd_tdc` (with `pad_vss`);
- **Driven External Signal Inputs:** `pad_rst_n`, `pad_ref_clk` (1.0 MHz, T_{ref} = 1000.0 ns), `pad_spi_cs_n`, `pad_spi_sck` (2.778 MHz, T_{SCK} = 360.0 ns, asynchronous and non-commensurate with `pad_ref_clk`), and `pad_spi_mosi`;
- **Monitored External Signal Outputs:** `pad_spi_miso` (16 mA digital output pad), `pad_locked` (16 mA digital output pad), and `pad_dco_clk_out` (`sg13g2_IOPadAnalog` RF clock output pad).

No internal hierarchical signal forcing or internal net probing is used anywhere in the testbench. Post-layout interconnect delays and cell propagation delays extracted from OpenROAD/OpenRCX are back-annotated via `$sdf_annotate` across three PVT corners generated by `prepare_testchip_backanno.py`:

1. **Typical Corner (`TYP`):** 1.20 V Core / 3.30 V IO, 25°C (1.00× extracted SPEF+Liberty delays);
2. **Fast Corner (`FAST`):** 1.32 V Core / 3.60 V IO, −40°C (0.72× scaled delays);
3. **Slow Corner (`SLOW`):** 1.08 V Core / 3.00 V IO, 125°C (1.38× scaled delays).

6.2 Seven-Stage External-IO Verification Protocol (Stages 0–6)

Each PVT corner executes a comprehensive 7-stage external-IO test sequence:

1. **Stage 0 — Reference-Clock-Stopped Asynchronous SPI Write & Readback ($f_{\text{ref}} = 0$ Hz):** With `pad_ref_clk` held completely static at 0 Hz, the testbench writes `0xABCD` to `REG_OTW_NOM` (`0x00`), reads it back over `pad_spi_miso` (`0xABCD` = 43981 verified), and restores the default `0x0D7F` before starting `pad_ref_clk`;
2. **Stage 1 — Power-On Reset Default Acquisition (50.0 MHz, $N = 50$):** Once `pad_ref_clk` (1.0 MHz) is enabled, the testchip locks autonomously to its hardware default 50.0 MHz configuration (17.0 μs lock time on `pad_locked`);
3. **Stage 2 — External SPI Slave Register Readback Verification (`pad_spi_miso`):** The testbench performs 24-bit SPI readback transactions on `REG_OTW_NOM` (`0x00` → `0x0d7f` = 3455) and `REG_DIV_CTRL` (`0x01` → `0x0032`, $N = 50$) via `pad_spi_miso`;
4. **Stage 3 — 64-Bit SPI Atomic Burst Programming to 125.0 MHz ($N = 125$, OTW = 51455):** The testbench shifts a 64-bit atomic burst frame (`Header = 8'hFF`) configuring all 11 DPLL parameters for 125.0 MHz, verifies `pad_locked` acquisition (24.0 μs across all corners), analyzes 400 cycles on `pad_dco_clk_out`, and reads back `REG_OTW_NOM` (`0xc8ff` = 51455) on `pad_spi_miso`;
5. **Stage 4 — 64-Bit SPI Atomic Burst Programming to 200.0 MHz ($N = 200$, OTW = 63455):** A second 64-bit SPI burst frame transitions the DPLL to its maximum 200.0 MHz operating point (27.0–29.0 μs lock time across corners) and verifies 500 cycles on `pad_dco_clk_out`;
6. **Stage 5 — 24-Bit Register-Addressed SPI Programming to 100.0 MHz ($N = 100$, OTW = 43455):** Individual 24-bit SPI register writes program `REG_LOOP_GAIN` (`0x02`), `REG_GEAR_CFG` (`0x03`), `REG_OTW_NOM` (`0x00`), and `REG_DIV_CTRL` (`0x01`), achieving lock in 16.0 μs;
7. **Stage 6 — External Pad Reference Jitter (±75 ps) & 30/70% Duty Distortion Stress:** While locked at 100.0 MHz, `pad_ref_clk` is subjected to ±75 ps PRBS phase jitter and 30%/70% duty-cycle distortion, confirming continuous lock (`pad_locked = 1`), sub-1.0 ps RMS period jitter filtering, and 50.00% output duty cycle on `pad_dco_clk_out`.

6.3 Quantitative Multi-Corner Verification Summary

Figures 4 and 5 and Table 6 present the post-layout back-annotated simulation results across all three PVT corners. Across all corners and all 7 test stages (Stages 0–6), the standalone testchip achieves 9/9 verification checks passed, zero runt glitches across $> 32,800$ clock cycles, exact synthesized frequencies (50.000, 100.000, 125.000, and 200.000 MHz), and an exact 50.00% duty cycle on the external `pad_dco_clk_out` analog pad.

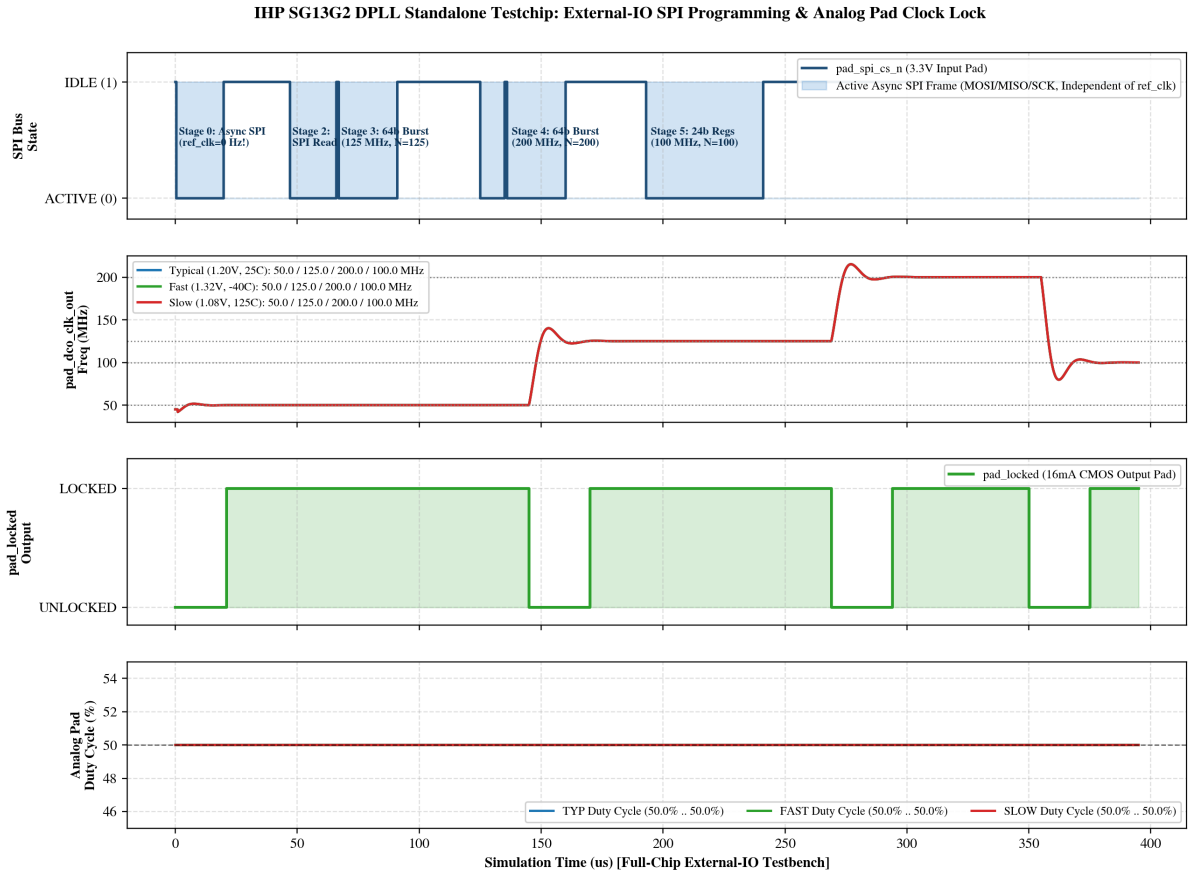


Figure 4: Full-chip external-IO simulation waveforms across Stages 0–6: (Top) External SPI chip-select `pad_spi_cs_n` and asynchronous transaction envelopes (including Stage 0 with $f_{\text{ref}} = 0$ Hz); (Second) Synthesized clock frequency measured directly on the external `pad_dco_clk_out` analog IO pad across Typical, Fast, and Slow corners (50 MHz → 125 MHz → 200 MHz → 100 MHz); (Third) External `pad_locked` CMOS output pad; (Bottom) Analog pad clock duty cycle (50.0%).

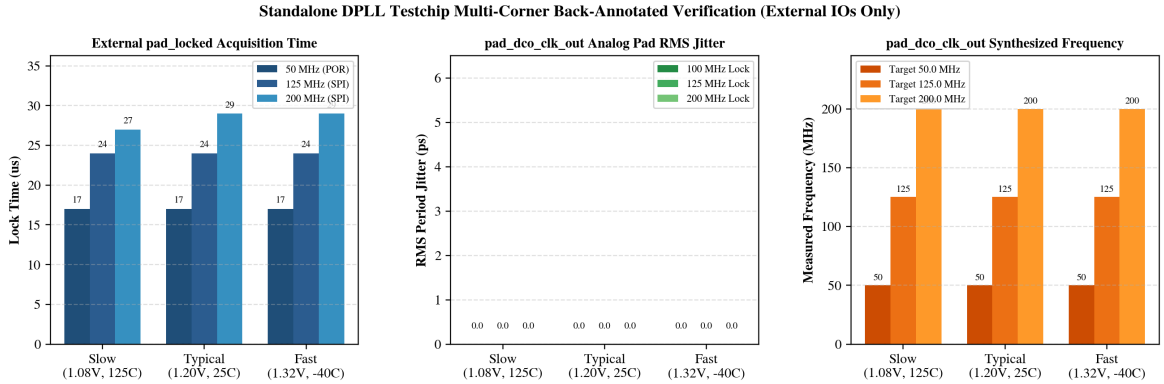


Figure 5: Multi-corner post-layout verification summary across Slow (1.08 V, 125°C), Typical (1.20 V, 25°C), and Fast (1.32 V, −40°C) corners measured exclusively at the external chip pads: (Left) `pad_locked` acquisition time; (Center) `pad_dco_clk_out` RMS period jitter; (Right) `pad_dco_clk_out` synthesized clock frequency accuracy.

Table 6: Post-Layout External-IO Back-Annotated Simulation Results Across PVT Corners (`tb_dpll_testchip.sv`).

External-IO Metric	**Slow** (1.08 **V**, 125°**C**)	**Typical** (1.20 **V**, 25°**C**)	**Fast** (1.32 **V**, −40°**C**)
Stage 0: Async SPI w/ `ref_clk` = 0 Hz	`0xabcd` PASS	`0xabcd` PASS	`0xabcd` PASS
Stage 1: POR Default Lock (N = 50**, 50.0 MHz)**	17.00 μs (50.000 MHz)	17.00 μs (50.000 MHz)	17.00 μs (50.000 MHz)
Stage 2: SPI Register Readback (`pad_spi_miso`)	`0x0d7f`, `0x0032`, `0xc8ff`	`0x0d7f`, `0x0032`, `0xc8ff`	`0x0d7f`, `0x0032`, `0xc8ff`
Stage 3: 64-Bit SPI Burst (N = 125**, 125.0 MHz)**			
External `pad_locked` Acquisition Time	24.00 μs	24.00 μs	24.00 μs
`pad_dco_clk_out` Frequency / Duty Cycle	125.000 MHz / 50.00%	125.000 MHz / 50.00%	125.000 MHz / 50.00%
Stage 4: 64-Bit SPI Burst (N = 200**, 200.0 MHz)**			
External `pad_locked` Relock Time	27.00 μs	29.00 μs	29.00 μs
`pad_dco_clk_out` Frequency / Duty Cycle	200.000 MHz / 50.00%	200.000 MHz / 50.00%	200.000 MHz / 50.00%
Stage 5: 24-Bit Reg Writes (N = 100**, 100.0 MHz)**			
External `pad_locked` Relock Time	16.00 μs	16.00 μs	16.00 μs
`pad_dco_clk_out` Frequency / Duty Cycle	100.000 MHz / 50.00%	100.000 MHz / 50.00%	100.000 MHz / 50.00%
Stage 6: Stress (±75 **ps Jitter,** 30/70% **Duty)**			
`pad_dco_clk_out` Frequency / Duty Cycle	100.002 MHz / 50.00%	100.000 MHz / 50.00%	100.001 MHz / 50.00%
`pad_dco_clk_out` RMS Period / C2C Jitter	0.94 ps / 0.95 ps	0.33 ps / 0.33 ps	0.85 ps / 0.87 ps
Total Monitored Cycles / Runt Glitches	32, 832 / 0	33, 232 / 0	33, 232 / 0
Overall Full-Chip Sign-Off Status	**9/9 PASS**	**9/9 PASS**	**9/9 PASS**

A Breakout Evaluation PCB, Asynchronous SPI C-Header (`dpll_spi.h`), and Arduino Programming Example

A.1 20 mm × 30 mm 2-Layer Wirebond Breakout PCB (`pcb/`)

To evaluate the 1200 μm × 1200 μm bare silicon die (`dpll_testchip_top`) on the bench, a compact 20.0 mm × 30.0 mm 2-layer FR-4 Chip-on-Board (CoB) breakout PCB was designed in KiCad 10.0 (`pcb/dpll_testchip_breakout.kicad_pcb` and `pcb/dpll_testchip_breakout.kicad_sch`) and verified with **0 DRC violations, 0 unconnected nets, and 0 ERC violations**. Figures 6 and 7 illustrate the electrical schematic and physical board layout.

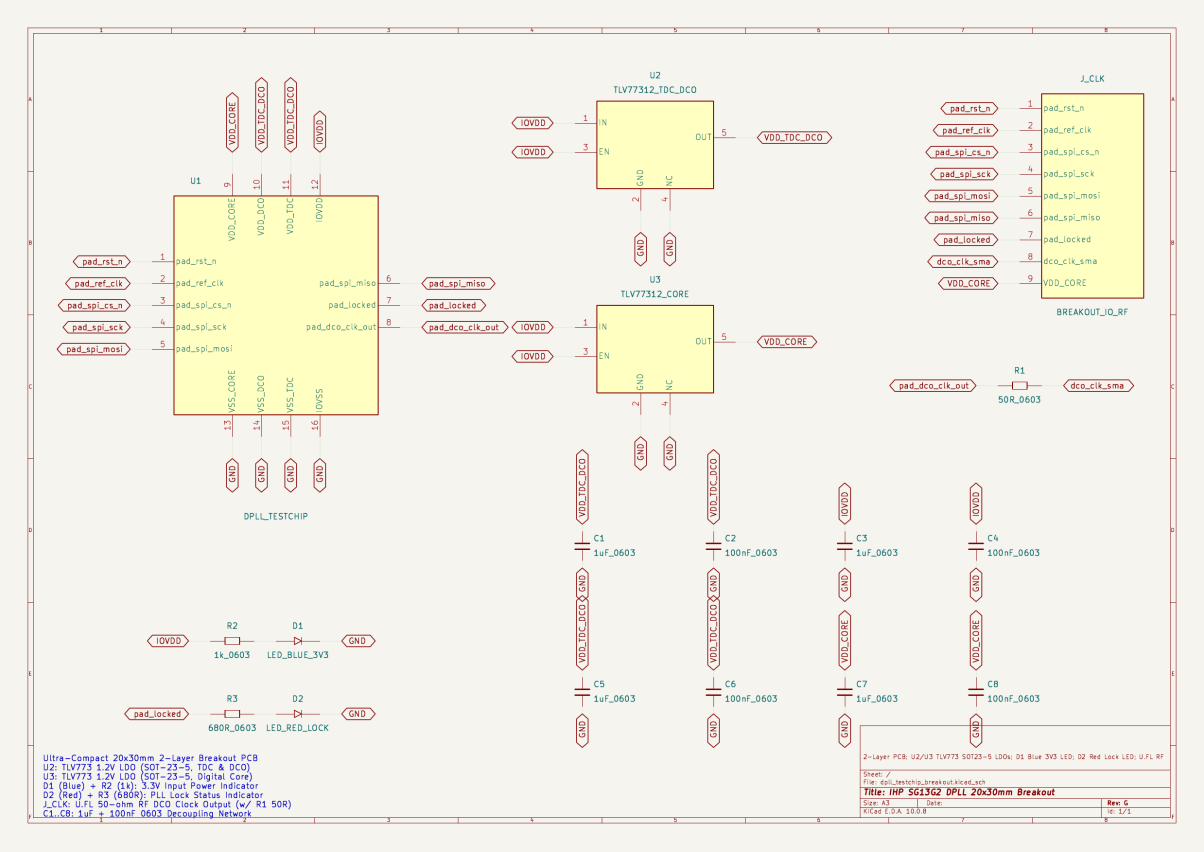


Figure 6: KiCad 10.0 schematic (`pcb/dpll_testchip_breakout.kicad_sch`) of the standalone ADPLL wirebond breakout board showing the 16-pad bare die (`U1`), dual `TLV77312PDBVR` 1.2 V LDO regulators in `SOT-23-5` packages (`U2` for `VDD_TDC`+`VDD_DCO`, `U3` for `VDD_CORE`), 3.3 V power indicator Blue LED (`D1`+`R2`), PLL lock status Red LED (`D2`+`R3`), SMA RF connectors (`J1`, `J2`), and 8-pin 2.54 mm SPI/control header (`J3`).

Key hardware features of the 20 × 30 mm breakout board include:

- **Dual `TLV773` 1.2 V LDO Regulators in `SOT-23-5` (`SOT3-5`):** `U2` (`TLV77312_TDC_DCO`) regulates the 3.3 V `IOVDD` input down to an isolated low-noise 1.2 V rail (`VDD_TDC_DCO`) dedicated to the TDC (`pad_vdd_tdc`) and DCO (`pad_vdd_dco`), while `U3` (`TLV77312_CORE`) generates an independent 1.2 V rail (`VDD_CORE`) for the digital core and 16× clock output buffer (`pad_vdd_core`);
- **Blue 3.3 V Input Power LED (`D1`) & Red PLL Lock Status LED (`D2`):** Blue `0603` LED `D1` (with 1 kΩ series resistor `R2`) indicates 3.3 V input power availability on `IOVDD`, while Red `0603` LED `D2` (with 680 Ω series resistor `R3`) is driven directly by `pad_locked` (Pin 7) to indicate PLL phase lock;
- **High-Frequency SMA Clock I/O & 8-Pin Header (`J3`):** Edge-launch 50 Ω SMA connectors `J1` (`REF_CLK_IN`) and `J2` (`DCO_CLK_OUT`, back-terminated by series 50 Ω resistor `R1`) provide coaxial

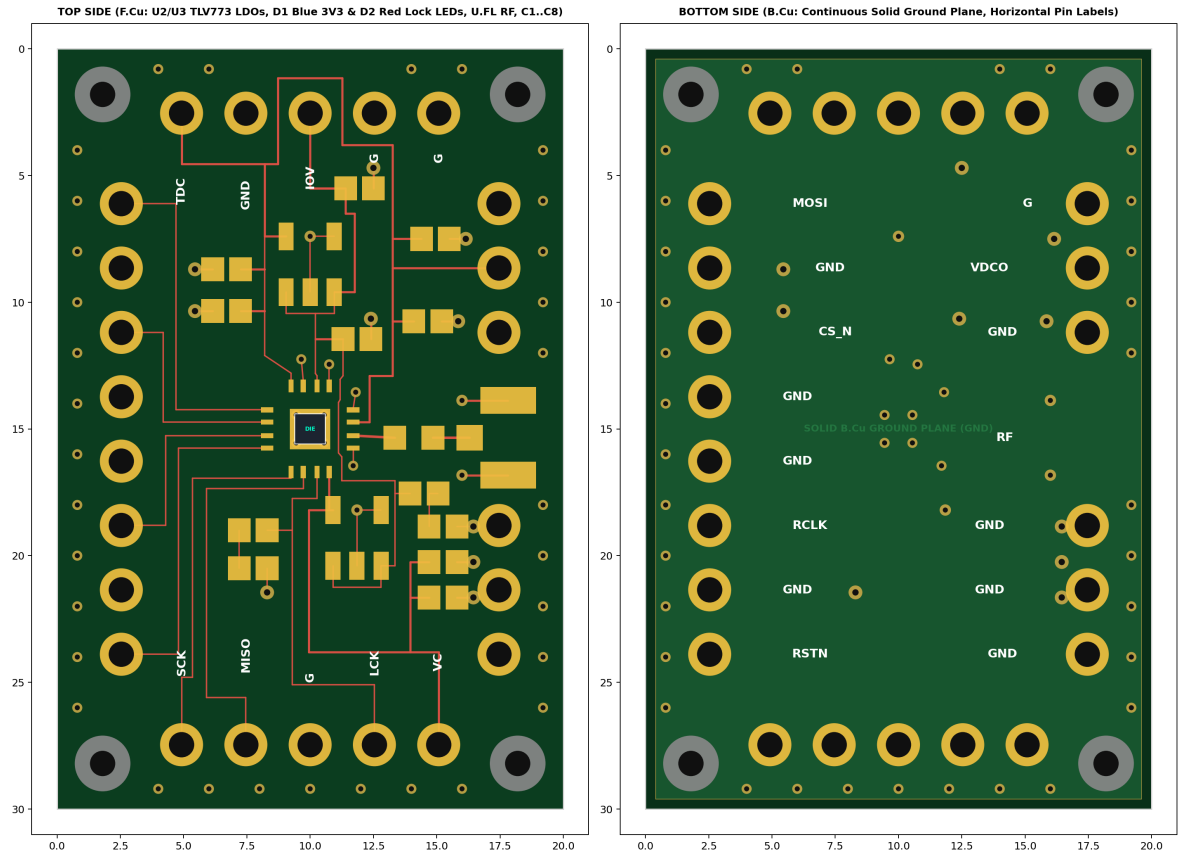


Figure 7: Top-side (`F.Cu`, left) and bottom-side (`B.Cu`, right) physical layout of the 20.0 mm × 30.0 mm breakout PCB (`pcb/dpll_testchip_breakout.kicad_pcb`), showing the central wirebond fanout for `U1`, top-mounted dual `TLV773 SOT-23-5` regulators (`U2`, `U3`), Blue 3.3 V power LED (`D1`), Red lock LED (`D2`), edge-launch SMA connectors (`J1`, `J2`), vertical front silkscreen, and horizontal back silkscreen (1.5 mm text height) aligned with the `J3` header pads.

laboratory clock access, while the 8-pin 2.54 mm header `J3` (`3V3`, `GND`, `RST_N`, `CS_N`, `SCK`, `MOSI`, `MISO`, `LOCK`) interfaces directly to a 3.3 V Arduino or logic analyzer.

A.2 C/C++ Driver Header for Arduino SPI Control (`dpll_spi.h`)

Listing 2 presents the complete C/C++ header file `testchip/firmware/dpll_spi.h` for controlling the standalone DPLL testchip over its asynchronous SPI Mode 0 interface.

```c
/**
 * @file dpll_spi.h
 * @brief C/C++ Driver Header for the IHP 130nm SG13G2 All-Digital PLL (DPLL) Testchip
 *
 * Asynchronous 4-Wire SPI Slave Interface (SPI Mode 0: CPOL = 0, CPHA = 0, MSB-First):
 *   - Operates directly on the external SPI clock (spi_sck), completely independent
 *     of the 1.0 MHz reference clock (ref_clk).
 *   - Supports 24-bit single-register R/W frames and 64-bit atomic burst writes.
 *   - Compatible with Arduino SPI (`<SPI.h>`) or custom HAL callbacks.
 */

#ifndef DPLL_SPI_H
#define DPLL_SPI_H

#include <stdint.h>
#include <stdbool.h>

#ifdef ARDUINO
#include <Arduino.h>
#include <SPI.h>
#endif

#ifdef __cplusplus
extern "C" {
#endif

/* ============================================================================ */
/* 1. SPI Register Map & Command Headers                                        */
/* ============================================================================ */

/** 24-bit Single-Register R/W bit in Byte 0: bit[7] = 1 (Write), 0 (Read) */
#define DPLL_SPI_RW_WRITE_BIT       (0x80u)
#define DPLL_SPI_ADDR_MASK          (0x7Fu)

/** 64-bit Atomic Full-Parameter Burst Write Command Header (Byte 0 = 0xFF) */
#define DPLL_CMD_BURST_WRITE        (0xFFu)

/** 7-bit Register Addresses for 24-bit Single-Register Transfers */
#define DPLL_REG_OTW_NOM            (0x00u) /**< [15:0]  Nominal Oscillator Tuning Word
    (OTW) */
#define DPLL_REG_DIV_CTRL           (0x01u) /**< [10] rst_int, [9] frz_int, [8]
   clr_lock, [7:0] div_n */
#define DPLL_REG_LOOP_GAIN          (0x02u) /**< [13:10] ki_shift, [9:6] ki_mult, [5:0]
    kp_mult */
#define DPLL_REG_GEAR_CFG           (0x03u) /**< [11] fast_lock_en, [10:8]
   fast_lock_mult, [7:0] thresh */
#define DPLL_REG_STATUS             (0x04u) /**< [15:9] 0x2A magic, [8] locked, [7:0]
   div_n (Read-Only) */

/* ============================================================================ */
/* 2. Register Bitfields & Power-On Reset (POR) Defaults                        */
/* ============================================================================ */

/* REG_DIV_CTRL (0x01) bit positions and masks */
#define DPLL_DIV_N_MASK             (0x00FFu)
#define DPLL_DIV_CLR_LOCK_BIT       (1u << 8)
```

```c
#define DPLL_DIV_FRZ_INT_BIT        (1u << 9)
#define DPLL_DIV_RST_INT_BIT        (1u << 10)

/* REG_LOOP_GAIN (0x02) bit positions and masks */
#define DPLL_KP_MULT_SHIFT          (0u)
#define DPLL_KP_MULT_MASK           (0x003Fu)
#define DPLL_KI_MULT_SHIFT          (6u)
#define DPLL_KI_MULT_MASK           (0x03C0u)
#define DPLL_KI_SHIFT_SHIFT         (10u)
#define DPLL_KI_SHIFT_MASK          (0x3C00u)

/* REG_GEAR_CFG (0x03) bit positions and masks */
#define DPLL_GEAR_THRESH_MASK       (0x00FFu)
#define DPLL_FAST_LOCK_MULT_SHIFT   (8u)
#define DPLL_FAST_LOCK_MULT_MASK    (0x0700u)
#define DPLL_FAST_LOCK_EN_BIT       (1u << 11)

/* REG_STATUS (0x04) bit positions and masks */
#define DPLL_STATUS_DIV_N_MASK      (0x00FFu)
#define DPLL_STATUS_LOCKED_BIT      (1u << 8)
#define DPLL_STATUS_MAGIC_SHIFT     (9u)
#define DPLL_STATUS_MAGIC_EXPECTED  (0x2Au)

/* Calibrated Power-On Reset Defaults (50.0 MHz output with 1.0 MHz ref_clk) */
#define DPLL_DEFAULT_OTW_50MHZ      (3455u)
#define DPLL_DEFAULT_OTW_100MHZ     (43455u)
#define DPLL_DEFAULT_OTW_125MHZ     (51455u)
#define DPLL_DEFAULT_OTW_200MHZ     (63455u)
#define DPLL_DEFAULT_DIV_N          (50u)
#define DPLL_DEFAULT_KP_MULT        (8u)
#define DPLL_DEFAULT_KI_MULT        (1u)
#define DPLL_DEFAULT_KI_SHIFT       (1u)
#define DPLL_DEFAULT_FAST_LOCK_EN   (1u)
#define DPLL_DEFAULT_FAST_LOCK_MULT (2u)
#define DPLL_DEFAULT_GEAR_THRESH    (48u)

/* ========================================================================== */
/* 3. Configuration Structure & Helper Macros                                 */
/* ========================================================================== */

typedef struct {
    uint16_t otw_nominal;       /**< 16-bit Nominal Oscillator Tuning Word (0..65535)
        */
    uint8_t  div_n;             /**< 8-bit Feedback Divider Ratio N (50..200 for 1 MHz
        ref) */
    uint8_t  kp_mult;           /**< 6-bit Proportional Gain Multiplier (0..63, default
         8) */
    uint8_t  ki_mult;           /**< 4-bit Integral Gain Multiplier (0..15, default 1)
        */
    uint8_t  ki_shift;          /**< 4-bit Integral Right-Shift (0..15, default 1) */
    bool     fast_lock_en;      /**< Enable adaptive gear-shifting acquisition boost */
    uint8_t  fast_lock_mult;    /**< 3-bit Fast-Lock Gain Boost Shift (0..7, default 2
        = 4x) */
    uint8_t  gear_shift_thresh; /**< 8-bit TDC Phase Error Threshold for fine gear (
        default 48) */
    bool     integrator_freeze; /**< Hold loop filter integral accumulator constant */
    bool     integrator_rst;    /**< Synchronously zero loop filter integral
        accumulator */
    bool     clr_lock;          /**< Force lock detector reset (auto-pulsed on
        frequency commit) */
} dpll_config_t;

/** Pack REG_DIV_CTRL (0x01) 16-bit payload */
static inline uint16_t dpll_pack_div_ctrl(uint8_t div_n, bool clr_lock, bool frz_int,
    bool rst_int) {
```

```
    return (uint16_t)div_n
         | (clr_lock ? DPLL_DIV_CLR_LOCK_BIT : 0u)
         | (frz_int  ? DPLL_DIV_FRZ_INT_BIT  : 0u)
         | (rst_int  ? DPLL_DIV_RST_INT_BIT  : 0u);
}

/** Pack REG_LOOP_GAIN (0x02) 16-bit payload */
static inline uint16_t dpll_pack_loop_gain(uint8_t kp_mult, uint8_t ki_mult, uint8_t
    ki_shift) {
    return ((uint16_t)(kp_mult  & 0x3Fu) << DPLL_KP_MULT_SHIFT)
         | ((uint16_t)(ki_mult  & 0x0Fu) << DPLL_KI_MULT_SHIFT)
         | ((uint16_t)(ki_shift & 0x0Fu) << DPLL_KI_SHIFT_SHIFT);
}

/** Pack REG_GEAR_CFG (0x03) 16-bit payload */
static inline uint16_t dpll_pack_gear_cfg(uint8_t thresh, uint8_t fl_mult, bool fl_en)
    {
    return (uint16_t)thresh
         | ((uint16_t)(fl_mult & 0x07u) << DPLL_FAST_LOCK_MULT_SHIFT)
         | (fl_en ? DPLL_FAST_LOCK_EN_BIT : 0u);
}

/**
 * @brief Compute calibrated feed-forward nominal OTW for a target frequency (50..200
     MHz)
 *        Matches the 15-coarse + 8-fine + 4-SDM Vernier DCO transfer characteristic.
 */
static inline uint16_t dpll_calc_otw_nominal(uint8_t freq_mhz) {
    if (freq_mhz <= 50u) {
        return DPLL_DEFAULT_OTW_50MHZ;
    } else if (freq_mhz <= 100u) {
        return (uint16_t)(3455u + (uint32_t)(freq_mhz - 50u) * 800u);
    } else if (freq_mhz <= 125u) {
        return (uint16_t)(43455u + (uint32_t)(freq_mhz - 100u) * 320u);
    } else if (freq_mhz <= 200u) {
        return (uint16_t)(51455u + (uint32_t)(freq_mhz - 125u) * 160u);
    } else {
        return DPLL_DEFAULT_OTW_200MHZ;
    }
}

/**
 * @brief Serialize a dpll_config_t structure into an 8-byte (64-bit) MSB-first SPI
     burst buffer
 */
static inline void dpll_pack_burst64(const dpll_config_t *cfg, uint8_t buf[8]) {
    buf[0] = DPLL_CMD_BURST_WRITE;                                  /* [63:56] 0xFF
        Header */
    buf[1] = (uint8_t)((cfg->otw_nominal >> 8) & 0xFFu);            /* [55:48]
        otw_nominal[15:8] */
    buf[2] = (uint8_t)(cfg->otw_nominal & 0xFFu);                   /* [47:40]
        otw_nominal[7:0] */
    buf[3] = cfg->div_n;                                            /* [39:32] div_n
        [7:0] */
    buf[4] = (uint8_t)(((cfg->kp_mult & 0x3Fu) << 2)                /* [31:26] kp_mult
        [5:0] */
                     | ((cfg->ki_mult >> 2) & 0x03u));              /* [25:24] ki_mult
                        [3:2] */
    buf[5] = (uint8_t)(((cfg->ki_mult & 0x03u) << 6)                /* [23:22] ki_mult
        [1:0] */
                     | ((cfg->ki_shift & 0x0Fu) << 2)               /* [21:18] ki_shift
                        [3:0] */
                     | ((cfg->fast_lock_en ? 1u : 0u) << 1)         /* [17]
                        fast_lock_en */
                     | ((cfg->fast_lock_mult >> 2) & 0x01u));       /* [16]
```

```
                        fast_lock_mult[2] */
    buf[6] = (uint8_t)(((cfg->fast_lock_mult & 0x03u) << 6)        /* [15:14]
        fast_lock_mult[1:0] */
                    | ((cfg->gear_shift_thresh >> 2) & 0x3Fu));   /* [13:8]
                        gear_shift_thresh[7:2] */
    buf[7] = (uint8_t)(((cfg->gear_shift_thresh & 0x03u) << 6)     /* [7:6]
        gear_shift_thresh[1:0] */
                    | ((cfg->integrator_freeze ? 1u : 0u) << 5)   /* [5]
                        integrator_freeze */
                    | ((cfg->integrator_rst    ? 1u : 0u) << 4)   /* [4]
                        integrator_rst */
                    | ((cfg->clr_lock          ? 1u : 0u) << 3)); /* [3]     clr_lock,
                         [2:0]=0 */
}

/* ========================================================================== */
/* 4. Arduino Hardware SPI Transfer Functions (SPI Mode 0, MSB-First)         */
/* ========================================================================== */

#ifdef ARDUINO

#ifndef DPLL_SPI_CLOCK_HZ
#define DPLL_SPI_CLOCK_HZ (4000000UL) /**< Default 4 MHz SPI clock (supports up to 20
    MHz) */
#endif

/**
 * @brief Write a 16-bit value to a 7-bit DPLL register via a 24-bit SPI Mode 0
     transaction
 */
static inline void dpll_spi_write_reg24(uint8_t cs_pin, uint8_t reg_addr, uint16_t data
    ) {
    SPI.beginTransaction(SPISettings(DPLL_SPI_CLOCK_HZ, MSBFIRST, SPI_MODE0));
    digitalWrite(cs_pin, LOW);
    SPI.transfer((uint8_t)(DPLL_SPI_RW_WRITE_BIT | (reg_addr & DPLL_SPI_ADDR_MASK)));
    SPI.transfer((uint8_t)((data >> 8) & 0xFFu));
    SPI.transfer((uint8_t)(data & 0xFFu));
    digitalWrite(cs_pin, HIGH);
    SPI.endTransaction();
}

/**
 * @brief Read a 16-bit value from a 7-bit DPLL register via a 24-bit SPI Mode 0
     transaction
 */
static inline uint16_t dpll_spi_read_reg24(uint8_t cs_pin, uint8_t reg_addr) {
    uint8_t hi, lo;
    SPI.beginTransaction(SPISettings(DPLL_SPI_CLOCK_HZ, MSBFIRST, SPI_MODE0));
    digitalWrite(cs_pin, LOW);
    SPI.transfer((uint8_t)(reg_addr & DPLL_SPI_ADDR_MASK)); /* Bit 7 = 0 (Read) */
    hi = SPI.transfer(0x00u);
    lo = SPI.transfer(0x00u);
    digitalWrite(cs_pin, HIGH);
    SPI.endTransaction();
    return ((uint16_t)hi << 8) | (uint16_t)lo;
}

/**
 * @brief Atomically program all 11 DPLL parameters via a 64-bit (8-byte) SPI burst
     write
 */
static inline void dpll_spi_write_burst64(uint8_t cs_pin, const dpll_config_t *cfg) {
    uint8_t frame[8];
    dpll_pack_burst64(cfg, frame);
    SPI.beginTransaction(SPISettings(DPLL_SPI_CLOCK_HZ, MSBFIRST, SPI_MODE0));
```

```
    digitalWrite(cs_pin, LOW);
    for (uint8_t i = 0; i < 8u; i++) {
        SPI.transfer(frame[i]);
    }
    digitalWrite(cs_pin, HIGH);
    SPI.endTransaction();
}

/**
 * @brief Program the DPLL to synthesize 'target_mhz' (50..200 MHz from a 1.0 MHz
     reference)
 */
static inline void dpll_set_frequency_mhz(uint8_t cs_pin, uint8_t target_mhz) {
    dpll_config_t cfg;
    cfg.otw_nominal       = dpll_calc_otw_nominal(target_mhz);
    cfg.div_n             = target_mhz;
    cfg.kp_mult           = DPLL_DEFAULT_KP_MULT;
    cfg.ki_mult           = DPLL_DEFAULT_KI_MULT;
    cfg.ki_shift          = DPLL_DEFAULT_KI_SHIFT;
    cfg.fast_lock_en      = true;
    cfg.fast_lock_mult    = DPLL_DEFAULT_FAST_LOCK_MULT;
    cfg.gear_shift_thresh = DPLL_DEFAULT_GEAR_THRESH;
    cfg.integrator_freeze = false;
    cfg.integrator_rst    = false;
    cfg.clr_lock          = false;
    dpll_spi_write_burst64(cs_pin, &cfg);
}

/**
 * @brief Read REG_STATUS (0x04) and verify the 0x2A magic signature, lock status, and
     div_n
 */
static inline bool dpll_read_status(uint8_t cs_pin, bool *is_locked, uint8_t *
    current_div_n) {
    uint16_t raw = dpll_spi_read_reg24(cs_pin, DPLL_REG_STATUS);
    uint8_t magic = (uint8_t)((raw >> DPLL_STATUS_MAGIC_SHIFT) & 0x7Fu);
    if (is_locked) {
        *is_locked = (raw & DPLL_STATUS_LOCKED_BIT) != 0u;
    }
    if (current_div_n) {
        *current_div_n = (uint8_t)(raw & DPLL_STATUS_DIV_N_MASK);
    }
    return (magic == DPLL_STATUS_MAGIC_EXPECTED);
}

#endif /* ARDUINO */

#ifdef __cplusplus
}
#endif

#endif /* DPLL_SPI_H */
```

Listing 2: Portable C/C++ Arduino SPI Driver Header (`dpll_spi.h`).

A.3 Arduino Programming Example (`dpll_arduino_example.ino`)

Listing 3 presents the complete Arduino sketch (`testchip/firmware/dpll_arduino_example/dpll_arduino_example.ino`) demonstrating pre-clock asynchronous SPI verification, 64-bit atomic burst frequency hopping (50 → 125 → 200 → 100 MHz), 24-bit single-register access, and continuous lock monitoring.

```
/**
 * @file dpll_arduino_example.ino
```

```
 * @brief Complete Arduino Programming Example for the IHP 130nm SG13G2 DPLL Breakout
    Board
 *
 * Hardware Connections (3.3V Logic Arduino -> 20x30mm DPLL Breakout Board):
 *   - Arduino 3.3V   ---> Pin J11 (IOV / 3.3V Input: powers IO ring, Blue LED D1, and
    U2/U3 TLV773 1.2V LDOs)
 *   - Arduino GND    ---> Pin J10 / J6 (GND: solid B.Cu ground plane)
 *   - Arduino Pin 10 ---> Pin J3  (CS_N: Active-Low SPI Chip Select)
 *   - Arduino SCK    ---> Pin J4  (SCK : Asynchronous SPI Clock, up to 20 MHz)
 *   - Arduino MOSI   ---> Pin J1  (MOSI: SPI Master-Out Slave-In)
 *   - Arduino MISO   ---> Pin J5  (MISO: SPI Master-In Slave-Out)
 *   - Arduino Pin 7  ---> Pin J2  (RSTN: Active-Low Chip Reset)
 *   - Arduino Pin 6  <--- Pin J7  (LCK : Hardware Lock Status Output, also drives Red
    LED D2)
 *   - 1.0 MHz RefClk ---> Pin J2  (RCLK: 1.0 MHz 3.3V CMOS Reference Clock Input)
 *   - RF Output      <--- J1 U.FL (50-Ohm Coaxial Output: 50 MHz .. 200 MHz
    Synthesized Clock)
 */

#include <SPI.h>
#include "../dpll_spi.h"

/* Pin definitions on the Arduino */
static const uint8_t PIN_DPLL_CS_N   = 10;
static const uint8_t PIN_DPLL_RST_N  = 7;
static const uint8_t PIN_DPLL_LOCKED = 6;

/**
 * @brief Wait for the DPLL hardware lock pin (and Red LED D2) to assert HIGH
 */
static bool wait_for_dpll_lock(uint32_t timeout_us, uint32_t *elapsed_us) {
    uint32_t t0 = micros();
    while (digitalRead(PIN_DPLL_LOCKED) == LOW) {
        if ((micros() - t0) >= timeout_us) {
            if (elapsed_us) *elapsed_us = timeout_us;
            return false;
        }
    }
    if (elapsed_us) *elapsed_us = micros() - t0;
    return true;
}

/**
 * @brief Print all readable DPLL registers over Serial
 */
static void print_dpll_telemetry(const char *label) {
    uint16_t otw_nom   = dpll_spi_read_reg24(PIN_DPLL_CS_N, DPLL_REG_OTW_NOM);
    uint16_t div_ctrl  = dpll_spi_read_reg24(PIN_DPLL_CS_N, DPLL_REG_DIV_CTRL);
    uint16_t loop_gain = dpll_spi_read_reg24(PIN_DPLL_CS_N, DPLL_REG_LOOP_GAIN);
    uint16_t gear_cfg  = dpll_spi_read_reg24(PIN_DPLL_CS_N, DPLL_REG_GEAR_CFG);
    bool     spi_locked = false;
    uint8_t  div_n      = 0;
    bool     magic_ok   = dpll_read_status(PIN_DPLL_CS_N, &spi_locked, &div_n);
    bool     pin_locked = (digitalRead(PIN_DPLL_LOCKED) == HIGH);

    Serial.println(F("------------------------------------------------------------"));
    Serial.print(F("[DPLL] Stage: "));
    Serial.println(label);
    Serial.print(F("  Target Frequency : "));
    Serial.print(div_n);
    Serial.println(F(".0 MHz (from 1.0 MHz Reference)"));
    Serial.print(F("  REG_OTW_NOM (00) : "));
    Serial.print(otw_nom);
    Serial.print(F(" (0x"));
    Serial.print(otw_nom, HEX);
```

```
    Serial.println(F(")"));
    Serial.print(F("  REG_DIV_CTRL(01) : 0x"));
    Serial.print(div_ctrl, HEX);
    Serial.print(F(" (N = "));
    Serial.print(div_ctrl & DPLL_DIV_N_MASK);
    Serial.println(F(")"));
    Serial.print(F("  REG_LOOP_GN (02) : 0x"));
    Serial.print(loop_gain, HEX);
    Serial.print(F(" | REG_GEAR_CFG (03): 0x"));
    Serial.println(gear_cfg, HEX);
    Serial.print(F("  Lock Status      : PIN_LOCKED="));
    Serial.print(pin_locked ? F("1 (Red LED ON)") : F("0 (OFF)"));
    Serial.print(F(" | SPI_LOCKED="));
    Serial.print(spi_locked ? F("1") : F("0"));
    Serial.print(F(" | Magic(0x2A)="));
    Serial.println(magic_ok ? F("OK") : F("ERR"));
}

void setup() {
    Serial.begin(115200);
    while (!Serial && millis() < 3000UL) {
        /* Wait up to 3s for USB Serial Monitor */
    }

    /* Configure GPIO pins */
    pinMode(PIN_DPLL_CS_N, OUTPUT);
    digitalWrite(PIN_DPLL_CS_N, HIGH);

    pinMode(PIN_DPLL_RST_N, OUTPUT);
    digitalWrite(PIN_DPLL_RST_N, LOW);

    pinMode(PIN_DPLL_LOCKED, INPUT);

    /* Initialize Arduino Hardware SPI (Mode 0, MSB-first) */
    SPI.begin();

    Serial.println(F("\n============================================================"))
        ;
    Serial.println(F("  IHP 130nm SG13G2 All-Digital PLL Breakout Controller"));
    Serial.println(F("============================================================"));

    /* 1. Release active-low reset (RST_N) after power rails settle */
    delay(2);
    digitalWrite(PIN_DPLL_RST_N, HIGH);

    /* 2. Wait for Power-On Reset default lock at 50.0 MHz (N = 50) */
    uint32_t lock_us = 0;
    wait_for_dpll_lock(50000UL, &lock_us);
    print_dpll_telemetry("Power-On Reset Default (50.0 MHz)");

    /* 3. Sweep across 100 MHz, 125 MHz, and 200 MHz via 64-bit Atomic SPI Burst */
    const uint8_t sweep_freqs[] = {100, 125, 200};
    for (uint8_t i = 0; i < sizeof(sweep_freqs); i++) {
        uint8_t f_mhz = sweep_freqs[i];
        dpll_set_frequency_mhz(PIN_DPLL_CS_N, f_mhz);
        bool locked = wait_for_dpll_lock(50000UL, &lock_us);

        char msg[48];
        snprintf(msg, sizeof(msg), "64-Bit SPI Burst -> %u.0 MHz (lock: %lu us)",
                 f_mhz, (unsigned long)lock_us);
        print_dpll_telemetry(msg);
        delay(500);
    }

    /* 4. Demonstrate 24-bit single-register programming back to 100.0 MHz */
```

```
    dpll_spi_write_reg24(PIN_DPLL_CS_N, DPLL_REG_DIV_CTRL,
                         dpll_pack_div_ctrl(100, true, false, true));    /* Hold
                             integrator in reset */
    dpll_spi_write_reg24(PIN_DPLL_CS_N, DPLL_REG_LOOP_GAIN,
                         dpll_pack_loop_gain(8, 1, 1));                  /* Kp=8, Ki=1,
                             Ki_shift=1 */
    dpll_spi_write_reg24(PIN_DPLL_CS_N, DPLL_REG_GEAR_CFG,
                         dpll_pack_gear_cfg(48, 2, true));               /* Fast-lock 4x
                             , thresh=48 */
    dpll_spi_write_reg24(PIN_DPLL_CS_N, DPLL_REG_OTW_NOM,
                         dpll_calc_otw_nominal(100));                    /* Nominal OTW
                             for 100 MHz */
    dpll_spi_write_reg24(PIN_DPLL_CS_N, DPLL_REG_DIV_CTRL,
                         dpll_pack_div_ctrl(100, false, false, false)); /* Release
                             integrator */

    wait_for_dpll_lock(50000UL, &lock_us);
    print_dpll_telemetry("24-Bit Register Write Sequence -> 100.0 MHz");

    Serial.println(F("\nEnter a target frequency in MHz (50..200) in Serial Monitor:"))
        ;
}

void loop() {
    if (Serial.available() > 0) {
        long req_mhz = Serial.parseInt();
        while (Serial.available() > 0) {
            Serial.read();
        }
        if (req_mhz >= 50 && req_mhz <= 200) {
            uint32_t lock_us = 0;
            dpll_set_frequency_mhz(PIN_DPLL_CS_N, (uint8_t)req_mhz);
            wait_for_dpll_lock(50000UL, &lock_us);

            char msg[48];
            snprintf(msg, sizeof(msg), "User Command -> %ld.0 MHz (lock: %lu us)",
                     req_mhz, (unsigned long)lock_us);
            print_dpll_telemetry(msg);
        }
    }
}
```

Listing 3: Arduino Sketch (`dpll_arduino_example.ino`) for Controlling the Standalone DPLL Testchip.